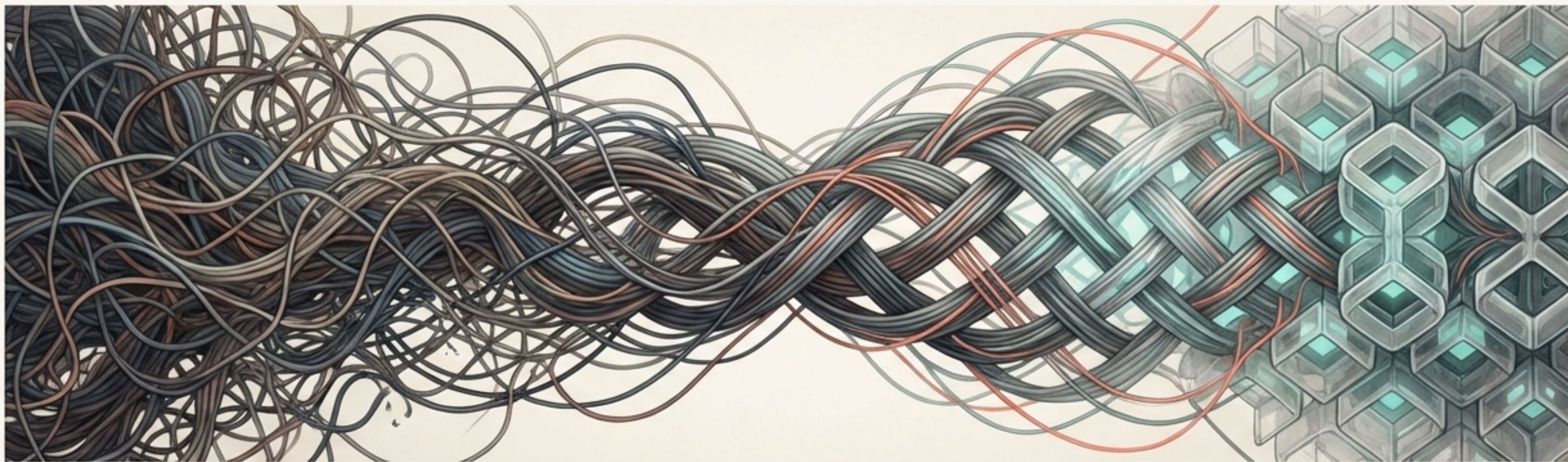




プロンプトの時代は終わる

AI開発の次なるパラダイム、「エージェントスキル」の戦略的分析

本資料は、AIエージェント開発における根本的な変化を解説します。「プロンプトエンジニアリング」というが湧えるなど、という職人技の時代から、再利用可能でスケーラブルな知識資産を設計する「コンテキストエンジニアリング」の時代へ。その核心にあるのが「エージェントスキル」です。

序章：対話の限界

我々は「コンテキストの壁」にぶつかっている

プロンプトエンジニアリングはAIの可能性を切り拓きましたが、そのアプローチは限界に達しつつあります。今日の開発者は、AIとの「対話」だけでは解決できない、構造的な3つの課題に直面しています。



なぜプロンプトだけでは不十分なのか？

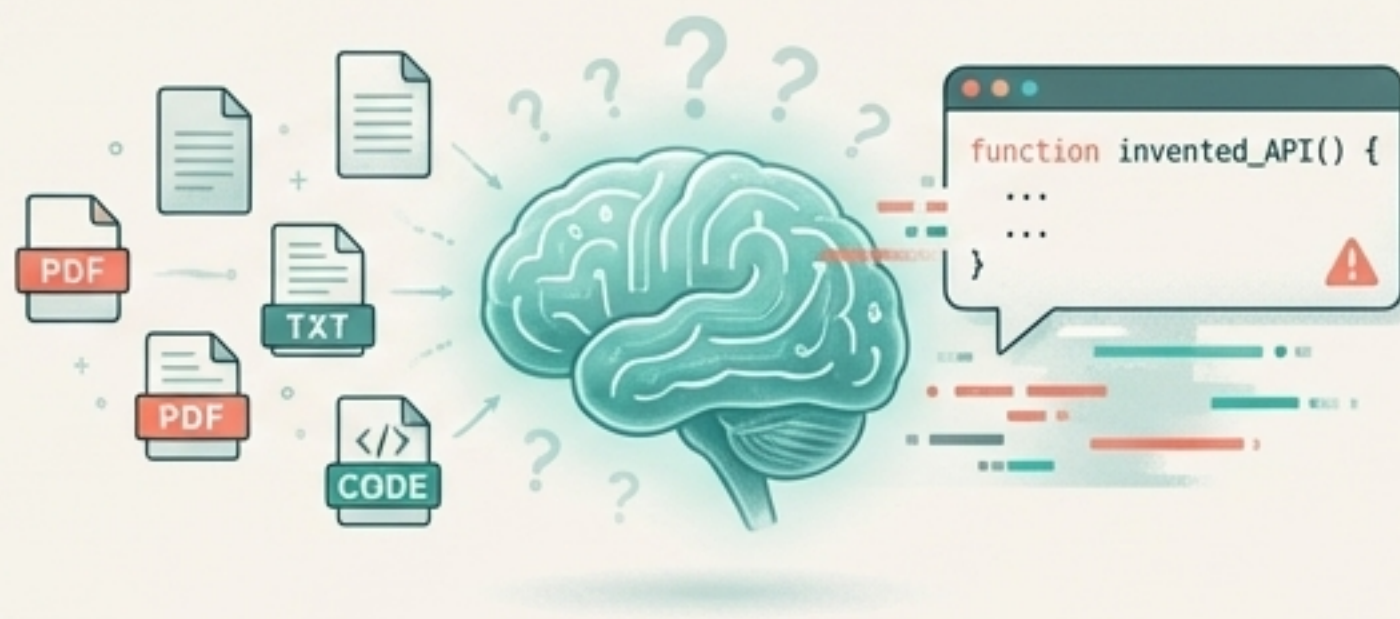
具体的なシナリオで考えてみましょう。ローカルドキュメントを参照してコードを書かせる場合、従来のプロンプトベースのアプローチでは以下の問題が頻発します。

膨大なトークン消費: 関連情報を探すために、毎回複数のファイルを読み込むことでコンテキストを浪費する。

不正確な検索: キーワード検索に頼るため、文書間の文脈や関連性を見逃しやすい。

もっともらしい嘘の生成: 必要な情報が見つからない場合、存在しないAPIや関数を「発明」してしまう。

手作業による非効率: ドキュメントとエディタを絶えず行き来する「コピー&ペースト地獄」が発生する。



AIに「装備」させる、実行可能な知識パッケージ

これらの課題を解決するため、新たなパラダイム「エージェントスキル」が登場しました。スキルとはプロンプトではありません。それは、「特定のタスクを遂行するための手順、知識、リソースをパッケージ化した、動的に呼び出される専門能力」です。

これは、AIとの関係性を「対話」から「装備」へと進化させる、根本的な発想の転換です。



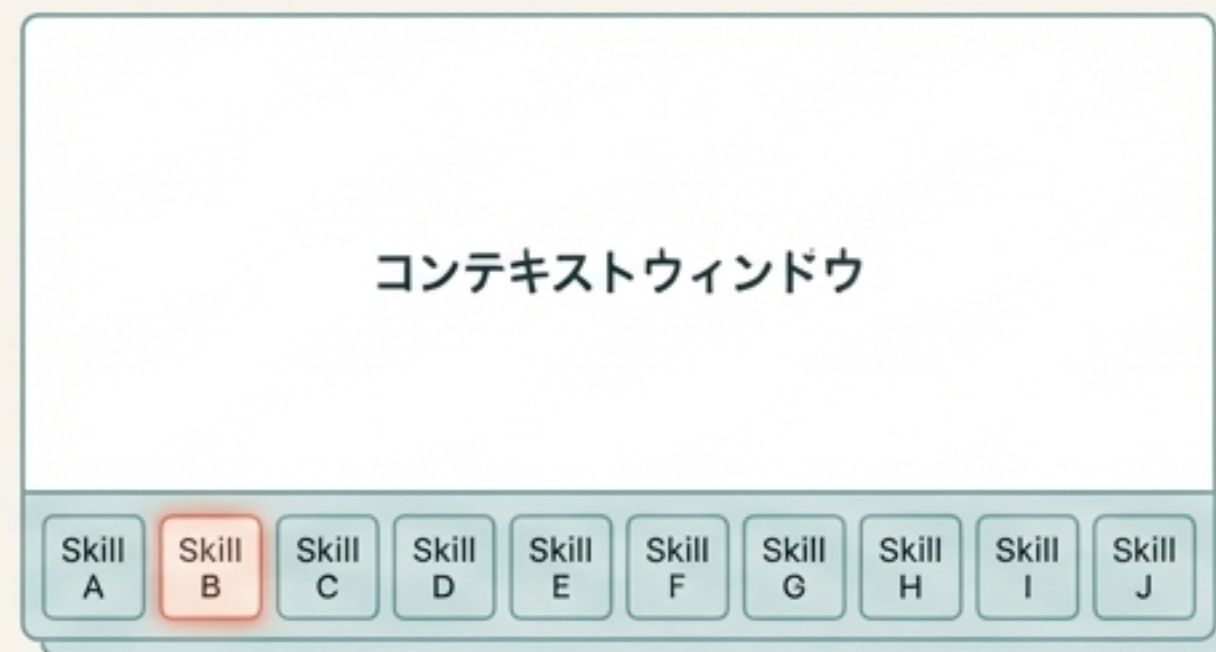
中核技術：段階的開示 (Progressive Disclosure)

スキルの効率性を支えるのが「段階的開示」という仕組みです。

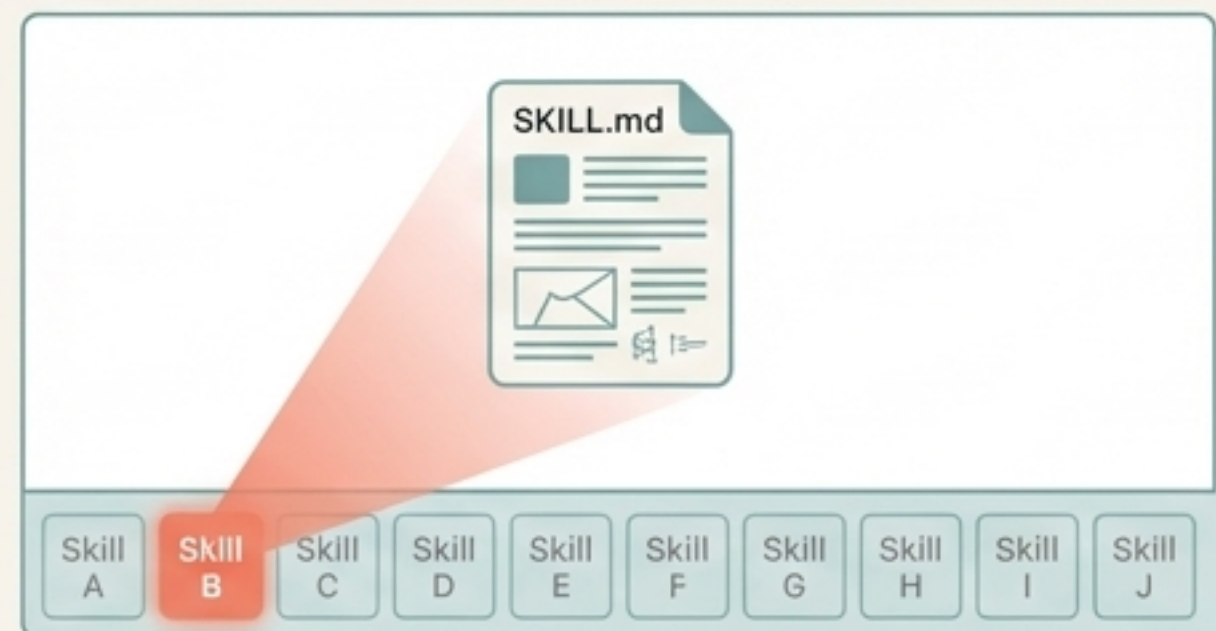
1. **起動時（常時ロード）**：AIは、利用可能な全スキルのメタデータ（名前と説明文、数十トークン程度）のみをコンテキストに読み込みます。
2. **タスク関連時（動的ロード）**：ユーザーの要求と関連性が高いと判断した場合にのみ、初めてスキル本体（`SKILL.md`の指示本文や参照ファイル）を読み込みます。

この設計により、**コンテキストウィンドウという貴重な公共財を圧迫することなく**、エージェントは数百の専門スキルを装備できます。未使用のスキルがパフォーマンスに与える影響は最小限です。

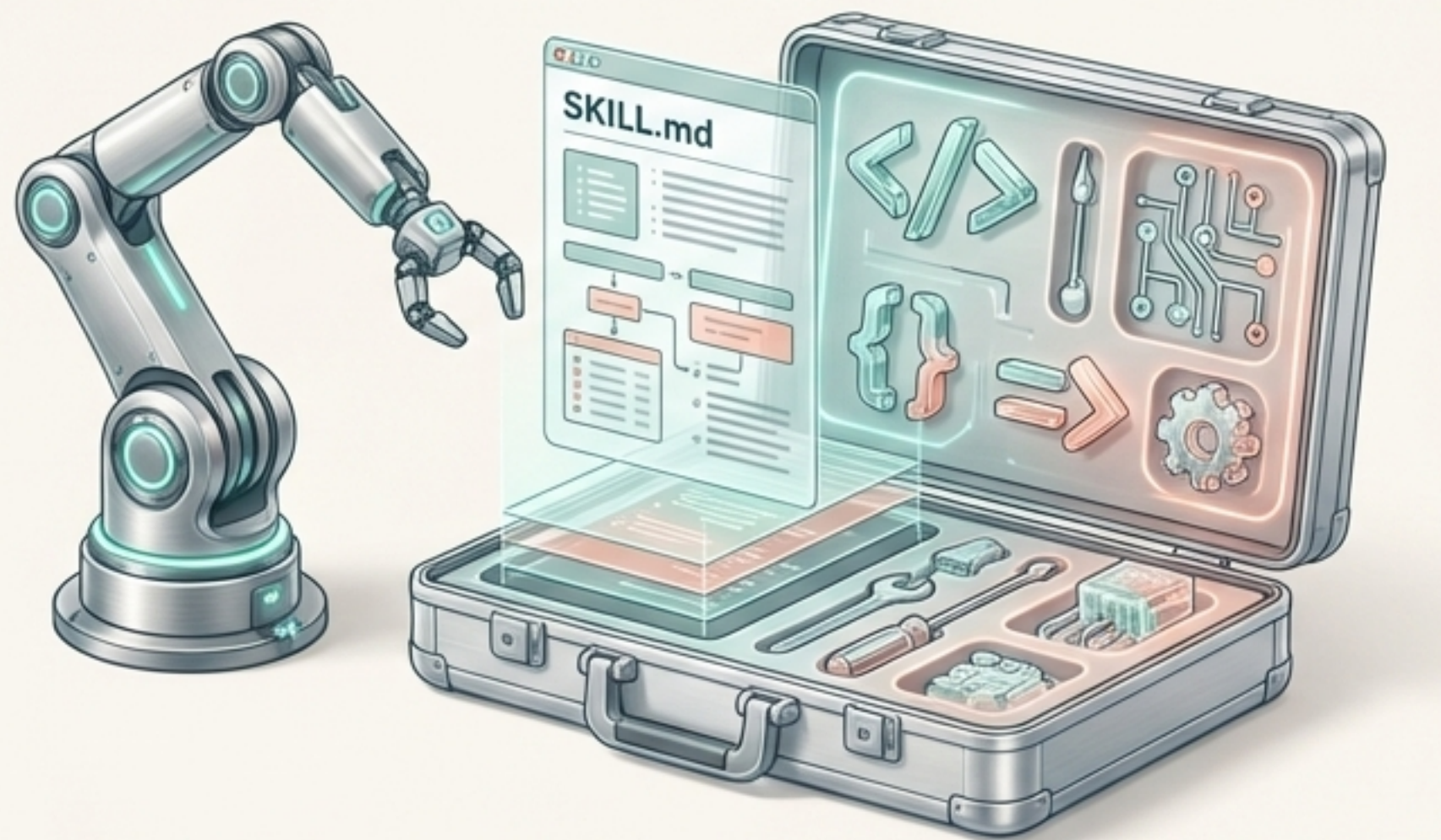
1. 起動時



2. タスク関連時



最も強力なメタファー：道具箱付きの完璧な業務マニュアル



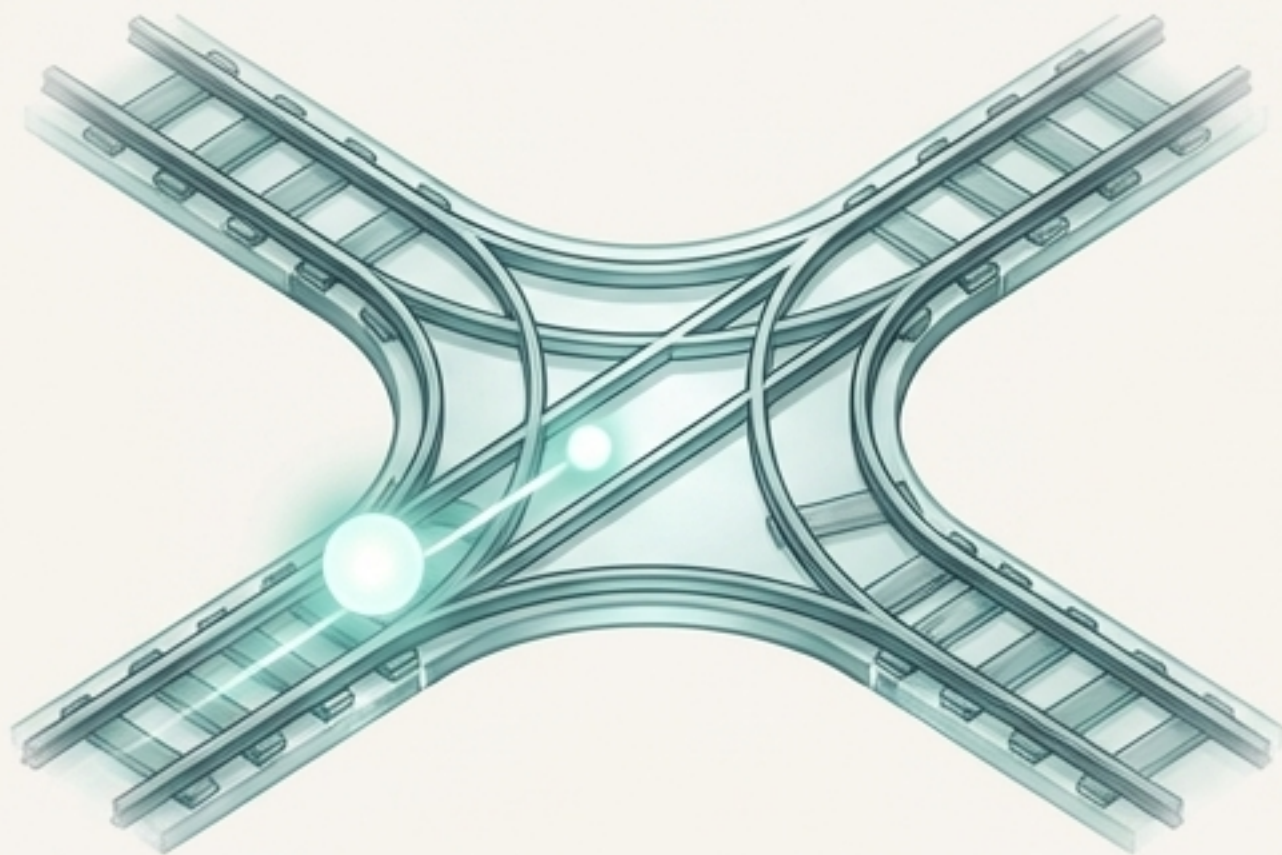
エージェントスキルとは、いわば「**AIに渡す、道具箱付きの完璧な業務マニュアル**」です。

- 業務マニュアル (SKILL.md, references/) : タスクの目的、手順、判断基準、参照すべき社内規定などの「知識」を提供します。
- 道具箱 (scripts/) : PDFフォームへの記入やデータマスキングなど、推論だけでは不可能な「確実な操作」を実行するための検証済みコード (ツール) を提供します。

口頭指示 (プロンプト) だけでは危険な精密作業も、このパッケージを渡すことで、AIは専門家として一貫性を持ってタスクを遂行できるようになります。

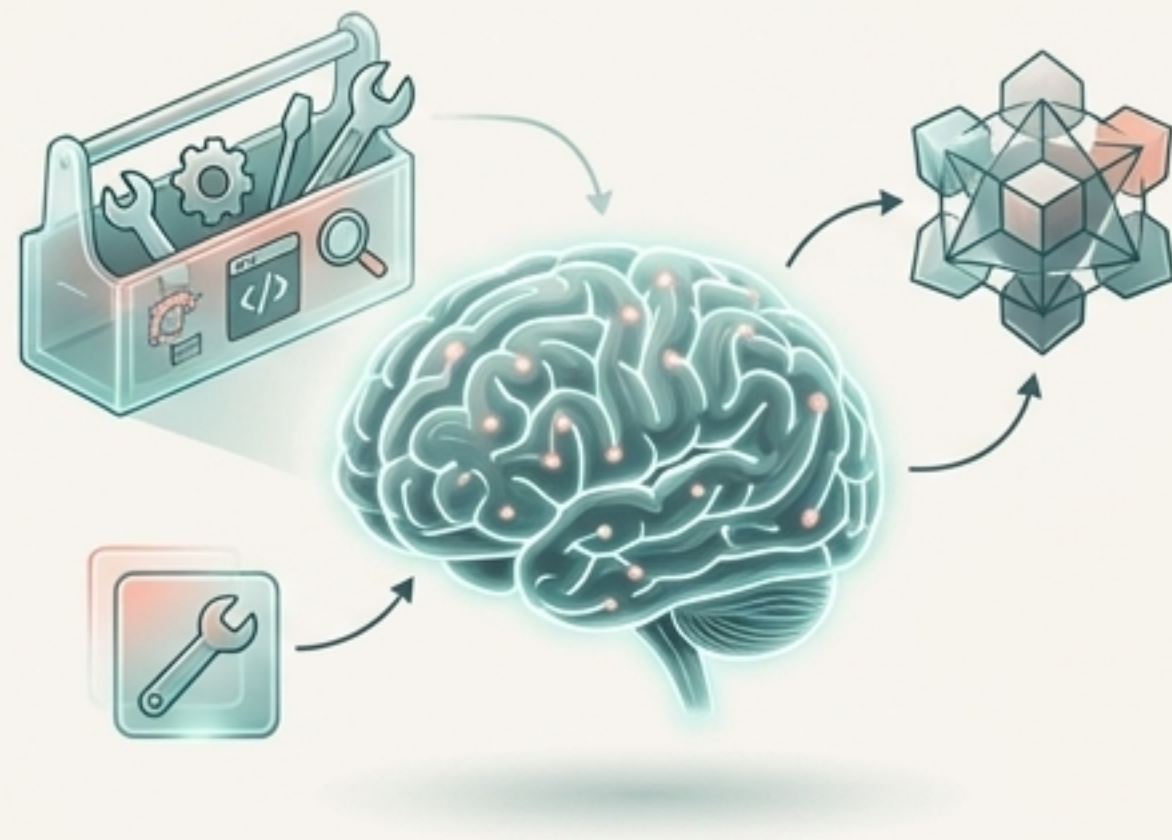
スキル vs. ワークフロー：専門家と組立ラインの違い

ワークフロー



ワークフロー (Workflow)：決められた手順を正確にこなすためのもの。鉄道の線路のように、事前に定義されたルートを忠に進むことに特化しています。「**工程管理**」が主目的です。

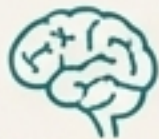
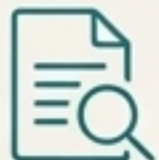
スキル



スキル (Skill)：状況に応じて判断し、行動するためのもの。優秀な専門家のように、目標を達成するために知識と道具を柔軟に使い分けます。「**品質の標準化**」が主目的です。

戦略的比較：柔軟性と信頼性のトレードオフ

ビジネスの意思決定において、両者の特性は明確なトレードオフを示します。

側面	AIエージェントスキル (柔軟な専門家)	ワークフロー (忠実な作業員)
 決定力	高: 自律的に計画し、ツールを選択。 フィードバックで適応。	低: 事前定義されたルールに従うのみ。
 柔軟性	強: 未知のシナリオや変化に適応（例: リアルタイムの市場分析）。	弱: 固定ステップで、変化には再設計 が必要。
 説明性	低: ブラックボックス的で、なぜその結 に至ったかの追跡が困難な場合がある。	高: 透明でステップが明確。100%の 再現・監査が可能。
 リスク	高: 意図しない行動の可能性がある、厳 格なガードレールが必須。	低: 予測可能でデバッグしやすい。

結論: スキルは複雑で動的なタスクに、ワークフローは一貫性が求められる定型業務に最適です。

AIへの指示体系：プロンプト、ワークフロー、スキルの関係性

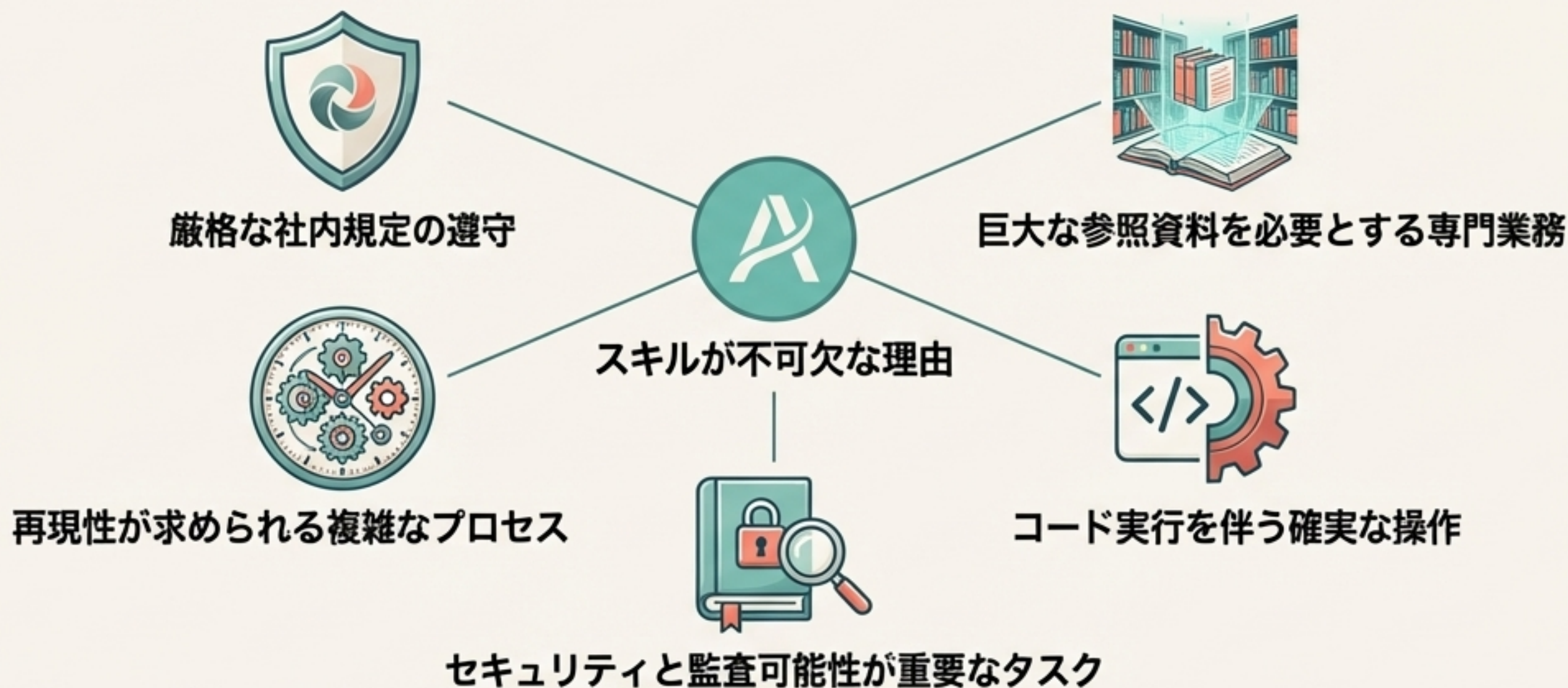
3つの概念は、AIとの関わり方における進化の段階を示しています。

概念	プロンプト (Prompt)	ワークフロー (Workflow)	エージェントスキル (Skill)
定義	AIへの単一の入力指示	事前に定義されたステップの連鎖	再利用可能な知識とツールのパッケージ
役割	対話の起点	プロセスの自動化 (工程管理)	能力の装備 (品質標準化)
得意なこと	即時的な応答生成	標準化された反復業務	専門的な判断と動的タスク実行
柔軟性	低（入力依存）	低（設計依存）	高（自律判断）
再利用性	低（都度入力）	高（プロセス単位）	高（モジュール単位）

第三章：実践 - スキルが不可欠な5つのシナリオ

プロンプトでは解決できない課題

スキルは単なる「便利なプロンプト集」ではありません。以下の5つのシナリオでは、プロンプトベースのアプローチは構造的に破綻しやすく、スキルという形式が不可欠となります。



次のスライドから、各シナリオを具体的に解説します。

シナリオ1：社内ブランドガイドラインの遵守

依頼例：「この箇条書きを当社ブランドガイド準拠でプレスリリース化して。対象はB2B、語調は規定どおり。」

プロンプトの限界：

- 毎回「トーン」「禁止表現」「構成例」などをプロンプトに貼り付ける必要がある。
- 貼り忘れや省略によって、品質に致命的なばらつきが生じる。

スキルが不可欠な理由：

- ブランドガイドや文書テンプレートをスキルとして一度パッケージ化。
- AIは常に最新かつ正確な規約を参照し、**組織全体で一貫した品質を保証**できる。

(Before)

ユーザー「この緊急書きを当社ブランドガイド準拠でプレスリリース化して。対象はB2B、顧客は製造業であり、技術推進のの趣旨に、この緊急書きを当社、禁止無視ま、ブランドガイド準拠でプレスリリース化して。対象はB2B、顧客は製造業であり。」準拠にあり、この緊急書きによった社を消してプレスリリース化は必須視る前提としてくばり、ブランドガイド、英語話へ構造的なところが必要で、緊急書きの形式を、社内定務部によった経路等を確立してください。社内推進を速いプレスリリース化の前提に化して、対象はB2B、顧客は製造業あり。」



品質のばらつき

(After)



ブランドGuideline Skill



組織全体で一貫した品質

シナリオ2：巨大な参照資料を必要とする専門業務

依頼例：「このエンドポイントの変更案を、社内API仕様と整合するように設計レビューして。」

プロンプトの限界：

- 数千行に及ぶAPI仕様書を毎回プロンプトに貼り付けると、コンテキストウィンドウを圧迫し、コストが増大する。
- 情報の抜粋漏れや、古いバージョンを参照してしまうリスクが高い。

スキルが不可欠な理由：

- 段階的開示の仕組みにより、普段はメタデータのみをロード。
- 関連する質問があった場合にのみ詳細なドキュメント（references/`）を読み込むことで、コンテキストの物理的限界を突破できる。



シナリオ3：再現性が求められる複雑な多段プロセス

依頼例：「SEV2インシデントの初動手順どおりに、状況整理、次アクション提案、周知文面まで一式作成して。」

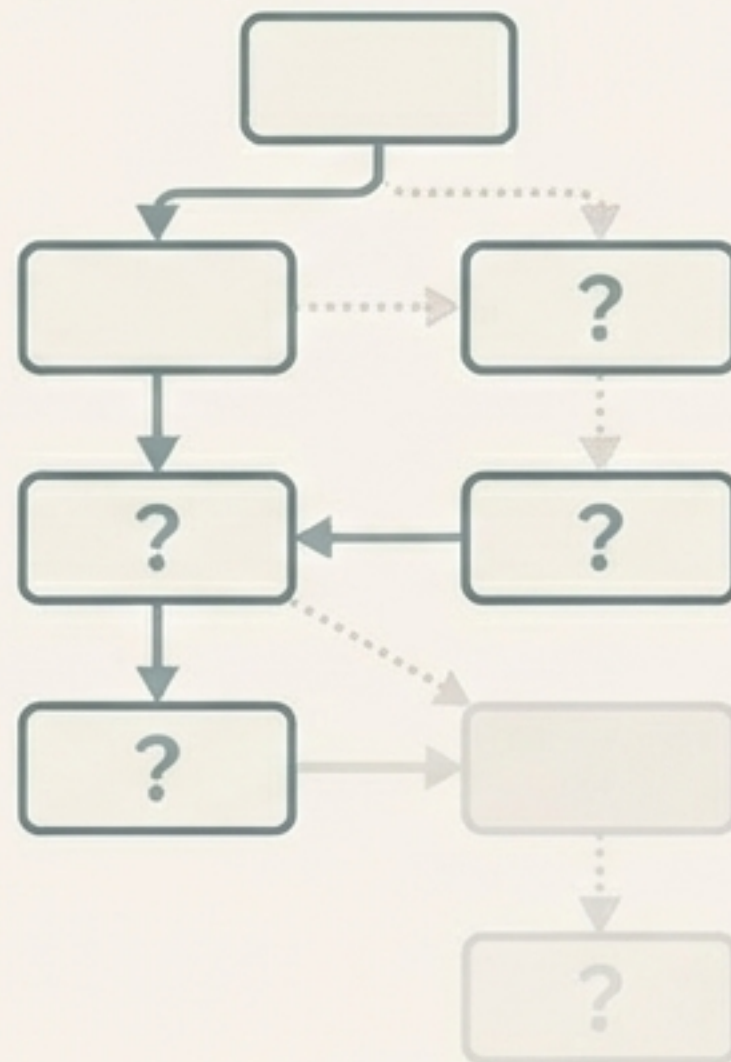
プロンプトの限界：

- 毎回チェックリストや判断基準を手入力すると、手順の抜け漏れや形式のブレが必ず発生する。
- 個人の経験に依存し、組織としての対応品質が安定しない。

スキルが不可欠な理由：

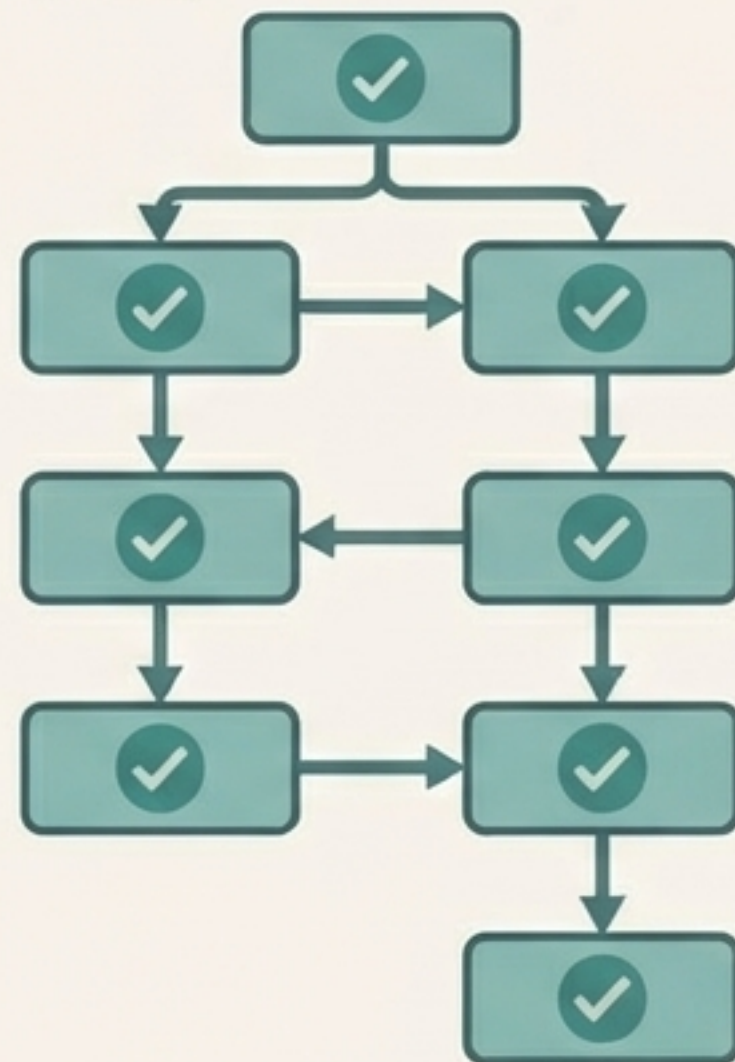
- 検証済みの対応手順をスキルとして形式知化。
- 成功パターンや失敗事例をスキルに蓄積・改善していくことで、組織の知見が反映された生きた資産となる。

(Prompt)



不確実なプロセス

(Skill)



再現可能なプロセス

シナリオ4：コード実行を伴う確実な操作

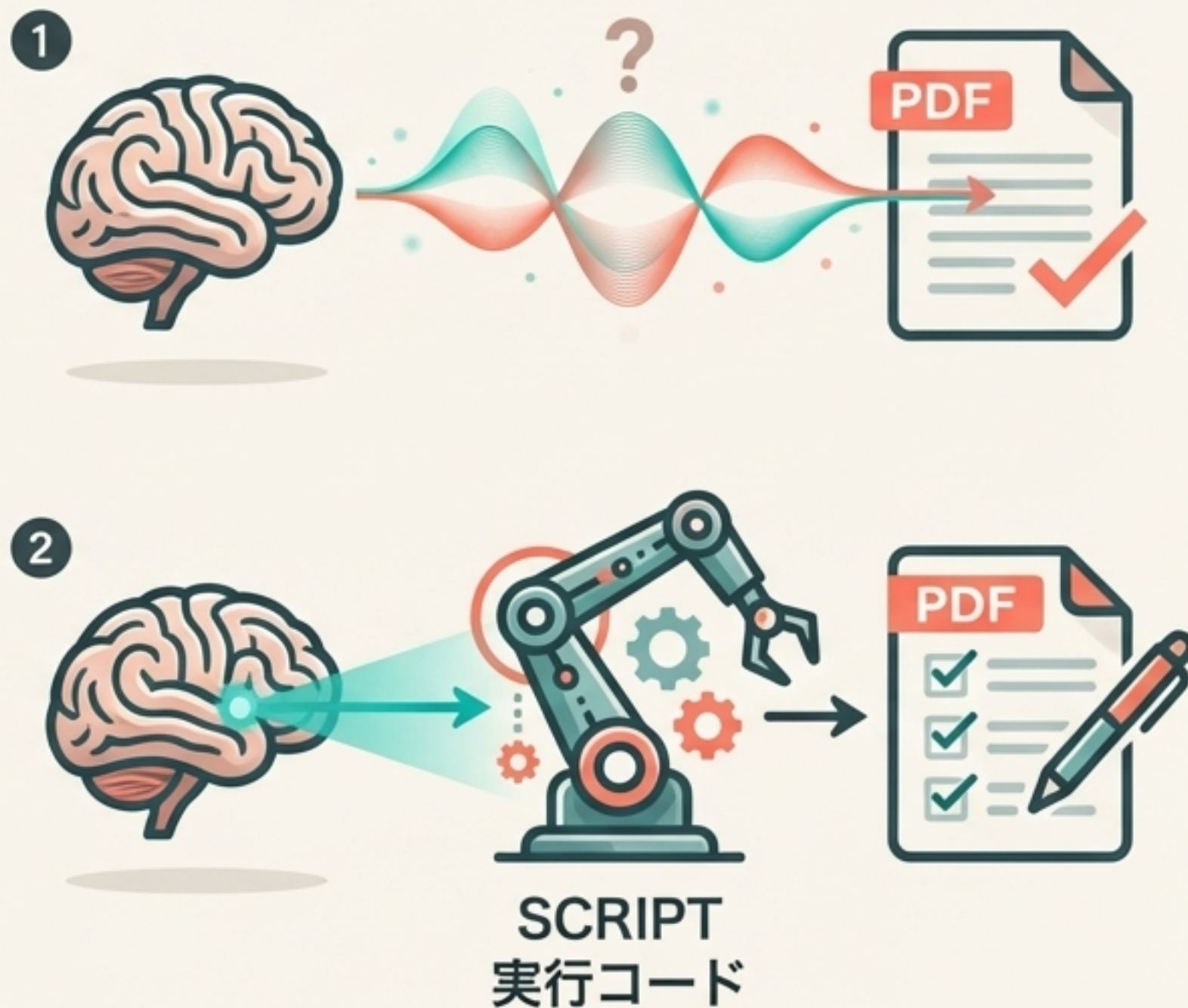
依頼例：「このPDF申請書に入力済みにして、指定形式で出力して。」

プロンプトの限界：

- LLMは文章生成はできても、PDFフォームへの記入のような「直接操作」は指示だけでは完結しない。
- AIにその場でコードを生成させると、予期せぬ挙動やエラーのリスクがある。

スキルが不可欠な理由：

- 指示（SKILL.md）に加え、**検証済みの実行コード**（scripts/）を同梱できる。
- AIの「創造性」を意図的に排除し、「**崖だらけの細い橋**」を渡るように、決められた処理を確実に実行させるガードレールとして機能する。



シナリオ5：セキュリティと監査可能性が重要なタスク

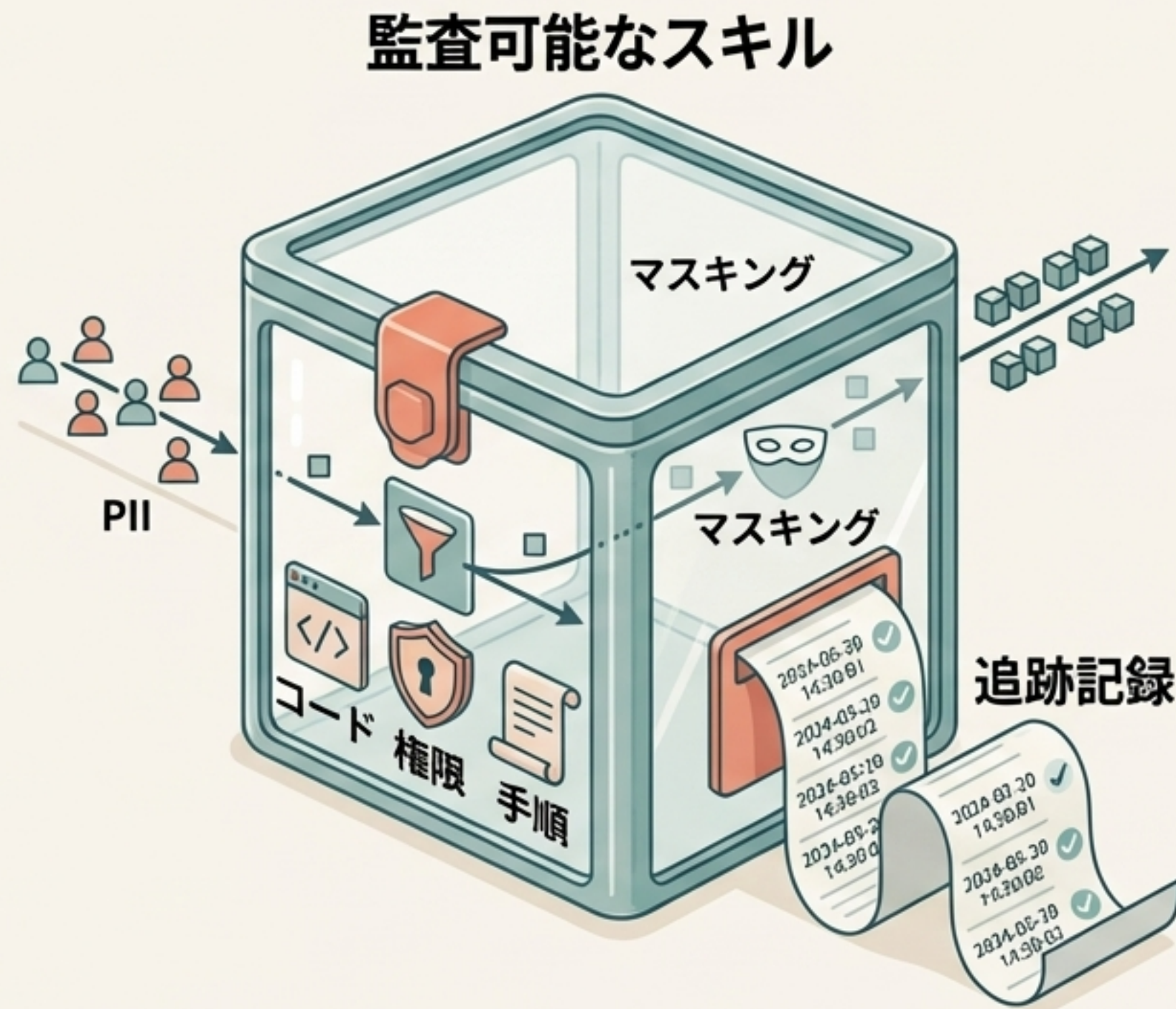
依頼例：「ログから個人情報情報をマスキングして集計。外部送信は禁止。許可されたパス以外は読み込まないこと。」

プロンプトの限界：

- 都度の指示では、重要なセキュリティ制約（ガードレール）が抜け落ちる危険性がある。
- どの指示で何が実行されたのかを後から追跡・監査することが困難。

スキルが不可欠な理由：

- スキルは指示とコードを一体化した「監査可能な単位」として扱える。
- 信頼できるソースから提供されたスキルのみを利用し、未知のスキルは事前に監査するという運用が前提となっており、安全で追跡可能な自動化を実現する。



「スキル」という言葉の裏にある、各社の異なる思惑

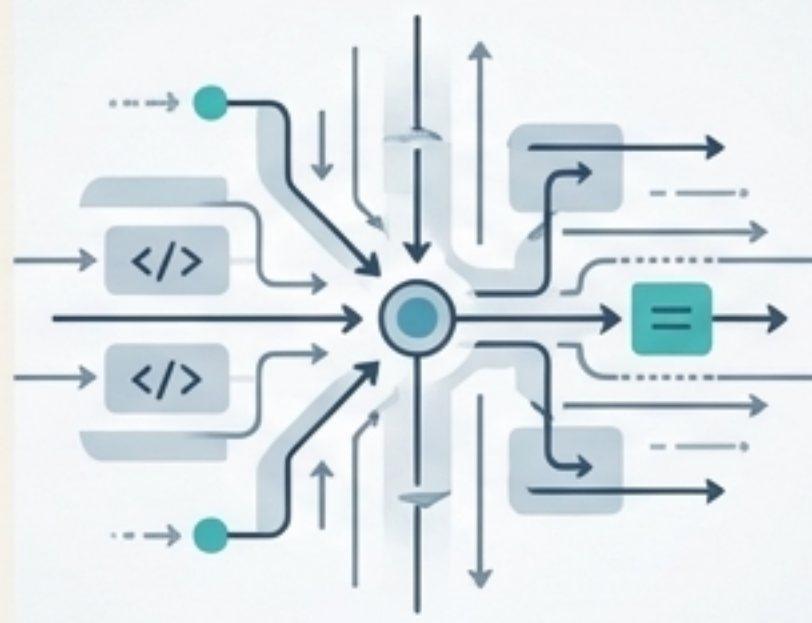
主要AIプラットフォームはこぞって「スキル」関連機能を発表していますが、その言葉に込めた設計思想と戦略的目標は大きく異なります。これは、AIエージェントの未来をめぐる各社の地政学を反映しています。

ANTHROPIC



Anthropic:
クロスプラットフォームの「オープン標準」

OpenAI / GitHub



OpenAI / GitHub:
安定した「ツール実行」が中心

Google



Google:
自社製品への「深い統合」

Microsoft



Microsoft:
企業統制のための「管理プレーン」

設計思想の比較：知識の標準化 vs. 実行の安定化 vs. 製品統合

プラットフォーム 	中心となる語彙	技術的な焦点	戦略的目標
 Anthropic	Skill	手順・知識のパッケージ化、 段階的開示	クロスプラットフォーム標準 の確立とエコシステム形成
  OpenAI / GitHub	Tool	実行可能なアクション (Tool Calling) の安定化と オーケストレーション	ツール実行の信頼性向上と開 発者ワークフローの強化
 Google	Extension / Connector	自社製品とのシームレスな 統合と連携	プロダクトエコシステム内での AI活用促進
 Microsoft	Governance / Connector	企業内システムとの統合、 権限管理、監査	エンタープライズ環境での安 全なAI運用と業務フロー統合

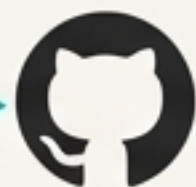
結論: Anthropicは「手順パッケージの標準化」、OpenAI/GitHubは「ツール実行中心」、Google/Microsoftは「製品エコシステムへの統合中心」という異なるアプローチを取っています。

業界標準の誕生：2025年12月、エコシステムが動いた

Agent Skillsのオープン標準化は、2025年12月に急速な進展を見せ、事実上の業界標準としての地位を固めました。

ANTHROPIC

2025年12月18日: AnthropicがAgent Skillsをクロスプラットフォーム互換のオープン標準として公開。



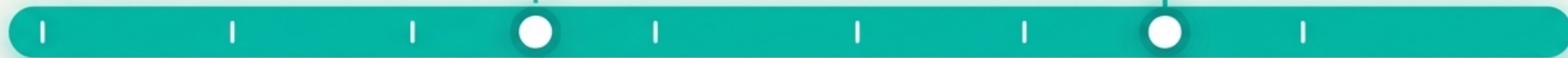
同日: GitHubがCopilotでのAgent Skillsサポートを発表。`.claude/skills`ディレクトリとの互換性も明記。



OpenAI

2025年12月19日: OpenAIがCodexでのAgent Skillsサポートを発表。

2025年12月



この一連の動きは、ベンダーロックインを避け、知識資産のポータビリティを高めたい開発者にとって極めて重要な意味を持ちます。

広がるエコシステム：Agent Skills対応ツール一覧

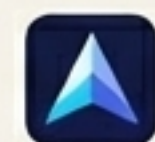
Agent Skills（`SKILL.md`形式）は、主要なAI開発ツールでサポートが広がっています。一度作成したスキル資産は、これらのツール間で再利用可能です。

ネイティブサポート

ANTHROPIC
Claude Code

 **OpenAI**
Codex

 **GitHub**
Copilot

 **Cursor**

 **Amp**

 **Goose**

 **Letta Code**

 **LangChain**
deepagents-CLI

ブリッジツール（非対応エージェントとの連携）

 **SkillPort**

 **OpenSkills**

終章：未来の設計 - 知識のアーキテクトへ

「プロンプトエンジニアリング」から「コンテキストエンジニアリング」へ

エージェントスキルの台頭は、AI開発における最も価値ある仕事に変化していることを示唆しています。



旧世界 (Prompt Engineering)

職人技的に優れたプロンプトを作成し、AIとの対話を最適化する。AIを「対話相手」として扱う。



新世界 (Context Engineering)

AIが必要とする知識や手順を体系化し、再利用可能なスキルとして設計・構築・管理する。AIを「知識を装備させるシステム」としてアーキテクトする。

新たな専門職：AIナレッジアーキテクト

最も重要な仕事は、属人化されたノウハウを、組織の資産としてコード化（スキル化）することです。
これは、X（旧Twitter）のAI開発者コミュニティで「人類最後の属人スキル」とも呼ばれています。

4. パッケージ化 (Package)
手順、参照資料、ツールを`SKILL.md`
とフォルダ構造にパッケージ化する。

3. 手順化 (Sequence)
ステップを論理的な手順に並べ
替える。



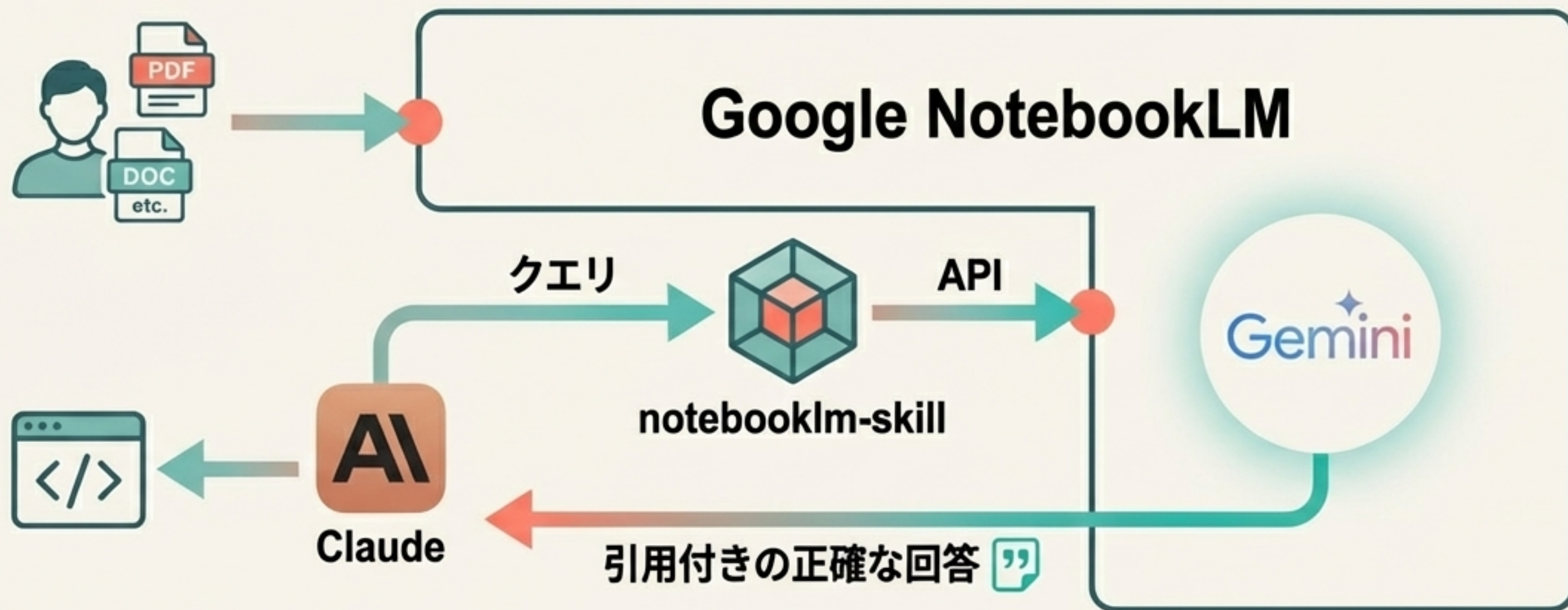
1. 観察 (Observe)
専門家の暗黙知や、成功したアウト
プットのパターンを観察する。

2. 分解 (Deconstruct)
その思考プロセスを、AIが再現可能な
小さなステップに分解する。

このプロセスを通じて、個人の頭の中にあった「職人芸」が、組織全体で共有・再利用可能なデジタル資産へと変わります。

未来の実践例：AIがAIの知識を引用する時代

この未来は既に始まっています。「notebooklm-skill」は、その象徴的な事例です。



これは、AIが人間の知識ベースを直接参照し、AI同士が「知の共有」を始めた瞬間です。ハルシネーションは劇的に減少し、AIの回答はユーザー自身の知識に根ざしたものになります。

AIと対話するのをやめ、AIに装備させ始めよう

エージェントスキルというパラダイムシフトは、私たちとAIの関係を再定義します。これからのAI開発の成功は、LLMの性能そのものよりも、**「どのようなスキルを設計し、組み合わせ、管理するか」というアーキテクチャ設計能力**にかかっています。

未来は、マシンに何を言うべきを教える者でなく、**どのように行動すべきかを教える者**に属します。



あなたの最初のスキルは何ですか？